

CSCE 649 PHYSICALLY-BASED MODELING · MODULE 06 · LECTURE NOTES

Collision Response

What to do once you know two things are touching.

MODULE OUTCOMES

- Derive the collision impulse with restitution.
- Implement Coulomb friction and explain the friction cone.
- Explain why resting contact is harder than collision.
- Implement sequential impulses with warm starting.
- Diagnose jitter, sinking, and drift in a contact solver.

1 Impulses

$$\mathbf{j} = -(\mathbf{1} + e) \mathbf{v}_{\text{rel}} / \left(\mathbf{1}/m_a + \mathbf{1}/m_b \right)$$

where $\mathbf{v}_{\text{rel}}$ is the relative velocity along the contact normal and e is the coefficient of restitution: 0 for a perfectly inelastic collision, 1 for a perfectly elastic one. The impulse is applied equally and oppositely, divided by each body's mass — so static geometry, with inverse mass zero, does not move.

WHY IMPULSES RATHER THAN FORCES

A real collision between rigid objects lasts on the order of microseconds and involves forces of enormous magnitude. Resolving it with forces would require a time step several orders of magnitude below anything affordable, for every collision in the scene.

An impulse is the integral of force over that brief interval, applied in a single instant. It changes velocity discontinuously, which is exactly what we observe at the scale we care about.

The consequence is that the deformation during the collision is not modelled at all. Everything that happens during the contact — elastic compression, plastic deformation, energy radiated as sound, heat — is summarised by a single number.

That is why the coefficient of restitution has no deep physical meaning. It is a lumped parameter standing in for a process we declined to simulate, which is why measured values depend on impact speed, temperature, and geometry in ways a single constant cannot capture.

2 Friction

$$|\mathbf{f}_t| \leq \mu |\mathbf{f}_n|$$

Regime	Condition	Behaviour
Static friction	The tangential force required to prevent sliding is within the bound.	The contact sticks. Relative tangential velocity is zero and friction supplies whatever force is needed, up to the limit.
Kinetic friction	The required force exceeds the bound.	The contact slides. Friction saturates at $\mu f_n $, opposing the relative motion.

The inequality is what makes friction difficult. Whether a given contact sticks or slides is not known in advance — it is an unknown that must be solved for. And the decisions are coupled: whether one contact slides affects the forces at others, which affects whether they slide. Friction cannot be resolved one contact at a time in a single pass.

THE FRICTION CONE AND THE PYRAMID EVERYONE SUBSTITUTES

The set of admissible contact forces — normal plus tangential — forms a **cone** about the contact normal, with half-angle $\arctan \mu$. Forces inside the cone are achievable; forces outside it are not, and the contact slides instead.

A cone is a nonlinear constraint, which complicates the solver considerably: the tangential bound depends on the normal force, which is itself being solved for simultaneously.

So almost every engine approximates the cone with a pyramid — four or eight flat sides — which reduces it to linear constraints the existing machinery already handles.

The artefact is slightly anisotropic friction. An object slides a little more easily along the pyramid's edge directions than along its face normals, so a box pushed across a floor drifts imperceptibly toward the diagonal. It is visible if you set up an experiment to look for it, and essentially nobody does — which is a reasonable trade, and worth knowing about.

3 Resting contact

RESTING CONTACT IS A DIFFERENT PROBLEM FROM COLLISION

A box sitting on the ground is not colliding. It is **in contact**, with zero relative normal velocity, and the ground must supply exactly enough upward force to cancel gravity — no more, which would launch it, and no less, which would let it sink.

Treating it as a sequence of tiny collisions produces jitter. Gravity pulls the box in during the step; an impulse pushes it out; the next step repeats. The box vibrates, and in a stack the vibration compounds upward.

And the forces are coupled through the stack. The force the ground exerts on the bottom box depends on the weight of every box above it, which depends on the forces between them. **You cannot solve the contacts independently** — the answer at one depends on the answer at all the others.

Formally this is a linear complementarity problem: at each contact, either the bodies are separating and the contact force is zero, or they are touching and the force is non-negative. Never both, and which case applies is part of the unknown.

```
for (int iter = 0; iter < iterations; iter++)
  for (auto& c : contacts) {
    double vn = relative_normal_velocity(c);
    double bias = BETA / dt * max(0.0, c.penetration - SLOP);
    double dj = -(vn + bias) / c.effective_mass;

    double old = c.normal_impulse;           // ACCUMULATED
    c.normal_impulse = max(0.0, old + dj);    // clamp the TOTAL
    dj = c.normal_impulse - old;             // apply the difference
    apply_impulse(c, dj * c.normal);
  }
```

CLAMP THE ACCUMULATED IMPULSE, NOT THE INCREMENT

Contacts may push but never pull, so the total normal impulse at a contact must be non-negative. The naive implementation clamps each iteration's increment to be non-negative, which is subtly and importantly wrong.

An individual iteration may legitimately need to reduce a previous over-correction. If an earlier iteration applied too large an impulse — because contacts solved later changed the situation — a later iteration must be able to subtract from it. Clamping the increment forbids that, and the solver cannot recover from its own overshoot.

Clamping the accumulated total instead permits negative increments while keeping the total non-negative, which is the correct condition.

This is the single most important detail in the whole solver, it appears in every production implementation, and the difference between the two versions is visible immediately in how well a stack converges.

Stabilisation method	Mechanism	Problem
Direct projection	Move the bodies apart to remove penetration.	Adds energy , and produces a visible pop.
Baumgarte stabilisation	Add a velocity bias proportional to penetration depth, so the solver naturally separates them.	Adds energy too, and behaves like a stiff spring — too large a coefficient makes stacks explode.
Split impulse	Run two passes: one for velocity, one for position correction, with the positional pass not feeding back into velocity.	No energy added. The correct answer, and what modern engines use.
Slop	Permit a small permanent overlap — a few millimetres — before correcting.	Essential rather than a hack. Without it, contacts oscillate between active and inactive every frame as the correction overshoots, and nothing settles.

4 Diagnosis

Symptom	Most likely cause
A stack jitters continuously.	Too few solver iterations, no warm starting , or no slop. Warm starting is the usual answer.
Objects sink into each other over time.	Insufficient iterations to propagate force through the stack; or position correction too weak.
Objects drift sideways while at rest.	Friction too weak, or the pyramid approximation's anisotropy.
Boxes creep slowly down a slope they should rest on.	Friction being solved without reference to the just-computed normal impulse. Solve the normal first, every iteration.
The stack explodes.	Baumgarte bias coefficient too large. It acts as a stiff spring; reduce it or switch to split impulse.
Objects never come fully to rest.	No sleeping. A settled object should be deactivated (Module 13).

WARM STARTING IS THE SINGLE BIGGEST LEVER

In a stack that is not moving, **the contact forces this frame are almost exactly the forces from last frame**. The answer barely changes.

Starting each solve from zero discards that information entirely and asks the solver to re-derive the whole force distribution from scratch, every frame, in ten iterations. For a tall stack it cannot, so the stack sags and jitters.

Store the accumulated impulse on each persistent contact point and apply it before the iterations begin. The solver then starts very close to the answer and converges in a few iterations.

This requires manifold persistence from Module 05 — contact points must be matched across frames by feature identity, not by position, so the stored impulse attaches to the right contact. The two mechanisms together are what make a tall, stable stack possible, and neither works without the other.

- **Solve friction after the normal impulse in each iteration**, since the friction bound $\mu|f_n|$ depends on the normal impulse just computed. Solving them in the other order, or in separate passes, gives noticeably worse friction.
- **Use a slop** of a few millimetres, scaled to the scene. See the table above.
- **Shock propagation for tall stacks**: solve contacts from the bottom upward, treating already-solved bodies as though they were static. Information propagates through the whole stack in one pass rather than one level per iteration. **This is the trick that makes hundred-box stacks feasible**.
- **Substep**. As in Modules 02 and 04, several small steps beat one large step with more iterations.
- **Sleep resting objects**. A stack that has settled should consume no time at all; Module 13 covers the hysteresis needed to do this without objects freezing mid-motion.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Erin Catto — Sequential Impulses and contact solver talks (GDC, free)	The method of §3, from its author. The clearest explanation of accumulated impulse clamping and warm starting anywhere.
Baraff — Rigid Body Simulation II: Nonpenetration Constraints (free notes)	The LCP formulation of resting contact, derived properly. Heavier going than Catto and worth it.
Mirtich — Impulse-based Dynamic Simulation (thesis, free)	The impulse formulation of §1 in full, including the friction cone treatment.
Box2D source	A production contact solver you can read in an afternoon. Compare your implementation against b2ContactSolver.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement impulse-based collision response with restitution. Verify energy behaviour for $e = 0$, 0.5 , and 1 — at $e = 1$ a bouncing ball must return to its original height.
- 2 Implement Coulomb friction with a pyramid approximation. Place a box on an inclined plane and find the angle at which it begins to slide; compare against $\arctan \mu$.
- 3 Demonstrate the pyramid artefact: push a box across a floor in many directions and plot the resulting trajectory deviation against direction.
- 4 Implement sequential impulses. Then change the clamp from accumulated to incremental and document how much worse the solver becomes.
- 5 Build a stack of ten boxes. Measure how long it stands with 4, 10, and 30 iterations.
- 6 Implement warm starting and repeat. Report the iteration count now needed for the same stability.
- 7 Sweep the Baumgarte coefficient and find the value at which the stack becomes springy, and the value at which it explodes.
- 8 Implement split impulse position correction and compare the energy behaviour against Baumgarte.
- 9 Implement shock propagation and build a stack of a hundred boxes. Report iterations and time against the non-shock version.
- 10 Produce the Project 1 deliverable: a ten-box stack standing for sixty seconds, with the energy plot showing it is genuinely at rest.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why are collisions resolved with impulses rather than forces?
- 2 What does the coefficient of restitution actually represent, and why is it not a fundamental constant?
- 3 Write the Coulomb friction condition and explain why the inequality makes it hard.
- 4 What is the friction cone, why is it approximated by a pyramid, and what is the artefact?
- 5 Why is resting contact not a sequence of collisions?
- 6 Why can contacts in a stack not be solved independently?
- 7 Why must the accumulated impulse be clamped rather than the increment?
- 8 Compare four penetration stabilisation methods.
- 9 What is warm starting, what does it require, and why does it matter so much?
- 10 Give four contact solver symptoms and their likely causes.

